There are several approaches for dealing with unknown future inputs. One approach is to form a probabilistic model of future inputs and design an algorithm that assumes future inputs conform to the model. This technique is common, for example, in the field of queuing theory, and it is also related to machine learning. Of course, you might not be able to develop a workable probabilistic model, or even if you can, some inputs might not conform to it. This chapter takes a different approach. Instead of assuming anything about the future input, we employ a conservative strategy of limiting how poor a solution any input can entail.

This chapter, therefore, adopts a worst-case approach, designing online algorithms that guarantee the quality of the solution for all possible future inputs. We'll analyze online algorithms by comparing the solution produced by the online algorithm with a solution produced by an optimal algorithm that knows the future inputs, and taking a worst-case ratio over all possible instances. We call this methodology ***competitive analysis***. We'll use a similar approach when we study approximation algorithms in Chapter 35, where we'll compare the solution returned by an algorithm that might be suboptimal with the value of the optimal solution, and determine a worst-case ratio over all possible instances.

We start with a "toy" problem: deciding between whether to take the elevator or the stairs. This problem will introduce the basic methodology of thinking about online algorithms and how to analyze them via competitive analysis. We will then look at two problems that use competitive analysis. The first is how to maintain a search list so that the access time is not too large, and the second is about strategies for deciding which cache blocks to evict from a cache or other kind of fast computer memory.

27.1 Waiting for an elevator

Our first example of an online algorithm models a problem that you likely have encountered yourself: whether you should wait for an elevator to arrive or just take the stairs. Suppose that you enter a

building and wish to visit an office that is k floors up. You have two choices: walk up the stairs or take the elevator. Let's assume, for convenience, that you can climb the stairs at the rate of one floor per minute. The elevator travels much faster than you can climb the stairs: it can ascend all k floors in just one minute. Your dilemma is that you do not know how long it will take for the elevator to arrive at the ground floor and pick you up. Should you take the elevator or the stairs? How do you decide?

Let's analyze the problem. Taking the stairs takes k minutes, no matter what. Suppose you know that the elevator takes at most $B - 1$ minutes to arrive for some value of B that is considerably higher than k. (The elevator could be going up when you call for it and then stop at several floors on its way down.) To keep things simple, let's also assume that the number of minutes for the elevator to arrive is an integer. Therefore, waiting for the elevator and taking it k floors up takes anywhere from one minute (if the elevator is already at the ground floor) to $(B - 1) + 1 = B$ minutes (the worst case). Although you know B and k, you don't know how long the elevator will take to arrive this time. You can use competitive analysis to inform your decision regarding whether to take the stairs or elevator. In the spirit of competitive analysis, you want to be sure that, no matter what the future brings (i.e., how long the elevator takes to arrive), you will not wait much longer than a seer who knows when the elevator will arrive.

Let us first consider what the seer would do. If the seer knows that the elevator is going to arrive in at most $k - 1$ minutes, the seer waits for the elevator, and otherwise, the seer takes the stairs. Letting m denote the number of minutes it takes for the elevator to arrive at the ground floor, we can express the time that the seer spends as the function

$$t(m) = \begin{cases} m + 1 & \text{if } m \le k - 1\,, \\ k & \text{if } m \ge k\,. \end{cases} \tag{27.1}$$

We typically evaluate online algorithms by their ***competitive ratio***. Let U denote the set (universe) of all possible inputs, and consider some input $I \in U$. For a minimization problem, such as the stairs-versus-

elevator problem, if an online algorithm A produces a solution with value $A(I)$ on input I and the solution from an algorithm F that knows the future has value $F(I)$ on the same input, then the competitive ratio of algorithm A is

$$\max \{A(I)/F(I) : I \in \mathcal{U}\}.$$

If an online algorithm has a competitive ratio of c, we say that it is ***c*-competitive**. The competitive ratio is always at least 1, so that we want an online algorithm with a competitive ratio as close to 1 as possible.

In the stairs-versus-elevator problem, the only input is the time for the elevator to arrive. Algorithm F knows this information, but an online algorithm has to make a decision without knowing when the elevator will arrive. Consider the algorithm "always take the stairs," which always takes exactly k minutes. Using equation (27.1), the competitive ratio is

$$\max \{k/t(m) : 0 \le m \le B-1\} . \tag{27.2}$$

Enumerating the terms in equation (27.2) gives the competitive ratio as

$$\max \left\{\frac{k}{1}, \frac{k}{2}, \frac{k}{3}, \ldots, \frac{k}{(k-1)}, \frac{k}{k}, \frac{k}{k}, \ldots, \frac{k}{k}\right\} = k ,$$

so that the competitive ratio is k. The maximum is achieved when the elevator arrives immediately. In this case, taking the stairs requires k minutes, but the optimal solution takes just 1 minute.

Now let's consider the opposite approach: "always take the elevator." If it takes m minutes for the elevator to arrive at the ground floor, then this algorithm will always take $m + 1$ minutes. Thus the competitive ratio becomes

$$\max \{(m+1)/t(m) : 0 \le m \le B-1\},$$

which we can again enumerate as

$$\max \left\{\frac{1}{1}, \frac{2}{2}, \ldots, \frac{k}{k}, \frac{k+1}{k}, \frac{k+2}{k}, \ldots, \frac{B}{k}\right\} = \frac{B}{k} .$$

Now the maximum is achieved when the elevator takes $B - 1$ minutes to arrive, compared with the optimal approach of taking the stairs, which requires k minutes.

Hence, the algorithm "always take the stairs" has competitive ratio k, and the algorithm "always take the elevator" has competitive ratio B/k. Because we prefer the algorithm with smaller competitive ratio, if $k = 10$ and $B = 300$, we prefer "always take the stairs," with competitive ratio 10, over "always take the elevator," with competitive ratio 30. Taking the stairs is not always better, or necessarily more often better. It's just that taking the stairs guards better against the worst-case future.

These two approaches of always taking the stairs and always taking the elevator are extreme solutions, however. Instead, you can "hedge your bets" and guard even better against a worst-case future. In particular, you can wait for the elevator for a while, and then if it doesn't arrive, take the stairs. How long is "a while"? Let's say that "a while" is k minutes. Then the time $h(m)$ required by this hedging strategy, as a function of the number m of minutes before the elevator arrives, is

$$h(m) = \begin{cases} m + 1 & \text{if } m \le k\,, \\ 2k & \text{if } m > k\,. \end{cases}$$

In the second case, $h(m) = 2k$ because you wait for k minutes and then climb the stairs for k minutes. The competitive ratio is now

$$\max \{h(m)/t(m) : 0 \le m \le B - 1\}\,.$$

Enumerating this ratio yields

$$\max \left\{\frac{1}{1}, \frac{2}{2}, \ldots, \frac{k}{k}, \frac{k+1}{k}, \frac{2k}{k}, \frac{2k}{k}, \frac{2k}{k}, \ldots, \frac{2k}{k}\right\} = 2\,.$$

The competitive ratio is now *independent* of k and B.

This example illustrates a common philosophy in online algorithms: we want an algorithm that guards against any possible worst case. Initially, waiting for the elevator guards against the case when the elevator arrives quickly, but eventually switching to the stairs guards against the case when the elevator takes a long time to arrive.

Exercises

27.1-1

Suppose that when hedging your bets, you wait for p minutes, instead of for k minutes, before taking the stairs. What is the competitive ratio as a function of p and k? How should you choose p to minimize the competitive ratio?

27.1-2

Imagine that you decide to take up downhill skiing. Suppose that a pair of skis costs r dollars to rent for a day and b dollars to buy, where $b > r$. If you knew in advance how many days you would ever ski, your decision whether to rent or buy would be easy. If you'll ski for at least $\lceil b/r \rceil$ days, then you should buy skis, and otherwise you should rent. This strategy minimizes the total that you ever spend. In reality, you don't know in advance how many days you'll eventually ski. Even after you have skied several times, you still don't know how many more times you'll ever ski. Yet you don't want to waste your money. Give and analyze an algorithm that has a competitive ratio of 2, that is, an algorithm guaranteeing that, no matter how many times you ski, you never spend more than twice what you would have spent if you knew from the outset how many times you'll ski.

27.1-3

In "concentration solitaire," a game for one person, you have n pairs of matching cards. The backs of the cards are all the same, but the fronts contain pictures of animals. One pair has pictures of aardvarks, one pair has pictures of bears, one pair has pictures of camels, and so on. At the start of the game, the cards are all placed face down. In each round, you can turn two cards face up to reveal their pictures. If the pictures match, then you remove that pair from the game. If they don't match, then you turn both of them over, hiding their pictures once again. The game ends when you have removed all n pairs, and your score is how many rounds you needed to do so. Suppose that you can remember the picture on every card that you have seen. Give an algorithm to play concentration solitaire that has a competitive ratio of 2.

27.2 Maintaining a search list